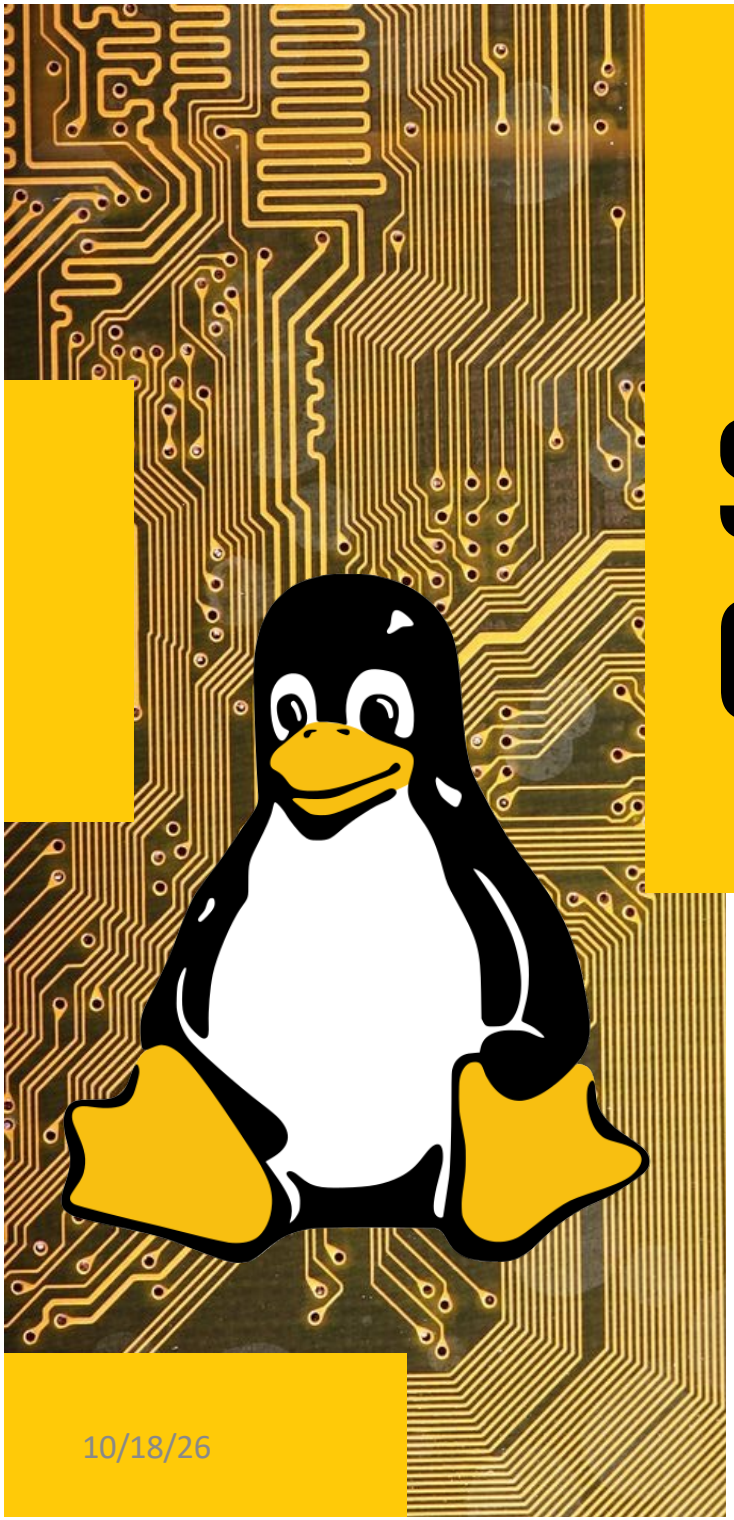


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

Synchronisation II

Readers and Writers, the Barber, the Philosophers

INSTRUCTOR: Jalolliddin Yusupov

Synchronisation II: Readers and Writers

Three problems with names

Readers and writers: many processes may read shared data at once, but only one at a time may change it. **The sleeping barber**: a barber who sleeps when nobody waits, and customers who leave when the chairs are full. **The dining philosophers**: five people, five forks, and everybody needs two.

Reading

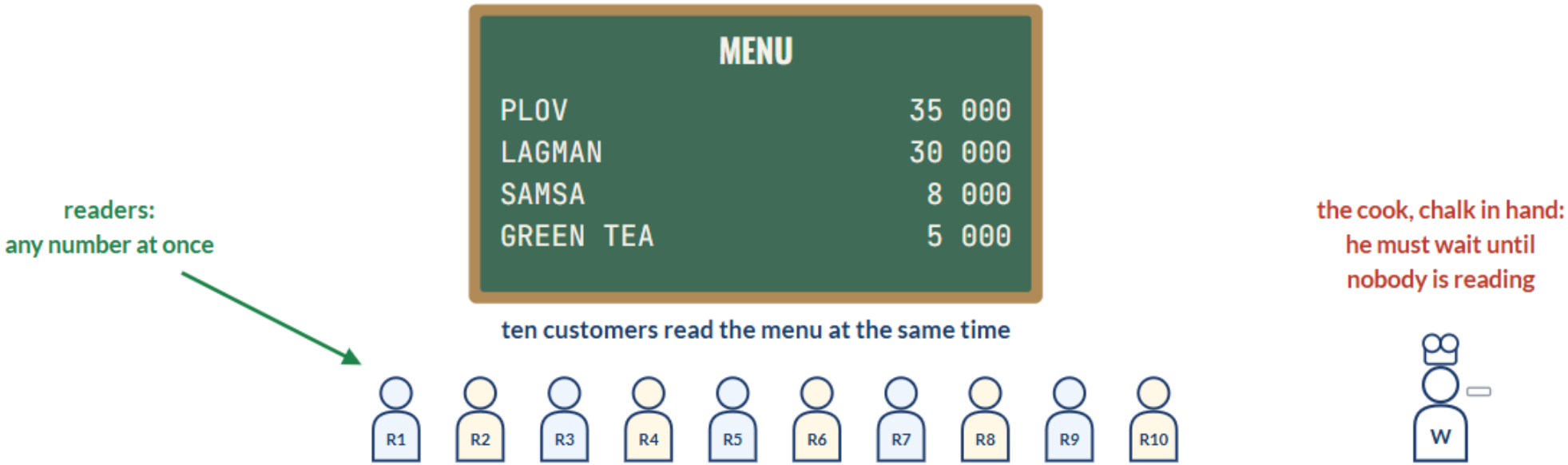
Allen B. Downey, *The Little Book of Semaphores*, ch. 4–6 (free online) · Silberschatz §7.1 and §8.3 · OSTEP ch. 31–32.

Already yours

`wait` and `post`, the trace table, the five jobs of a semaphore, `shared_mem.h`. One new C call, `sem_trywait` (take a flag if there is one, never sleep), is explained where it appears.



The Chalkboard: Readers and Writers



Reader and writer

A **reader** is a process that only **looks at** shared data (a customer reading the menu). A **writer** is one that **changes** it (the cook rewriting a price).

Why the writer must be alone

A customer who reads while the cook rewrites sees **half a price** — the first digit new, the rest old. Two cooks at once write over each other: Week 4's lost update, in chalk.

Why readers may share

Ten customers reading one board cannot hurt each other: nothing changes while they read.

The question of the day

One lock for the board would keep the readers and the cook apart. Is that good enough — and is it fair?

Any number of readers, or exactly one writer — never both.

Exercise: One Lock for the Board

```
board = Semaphore(1)     /* one key for the board */

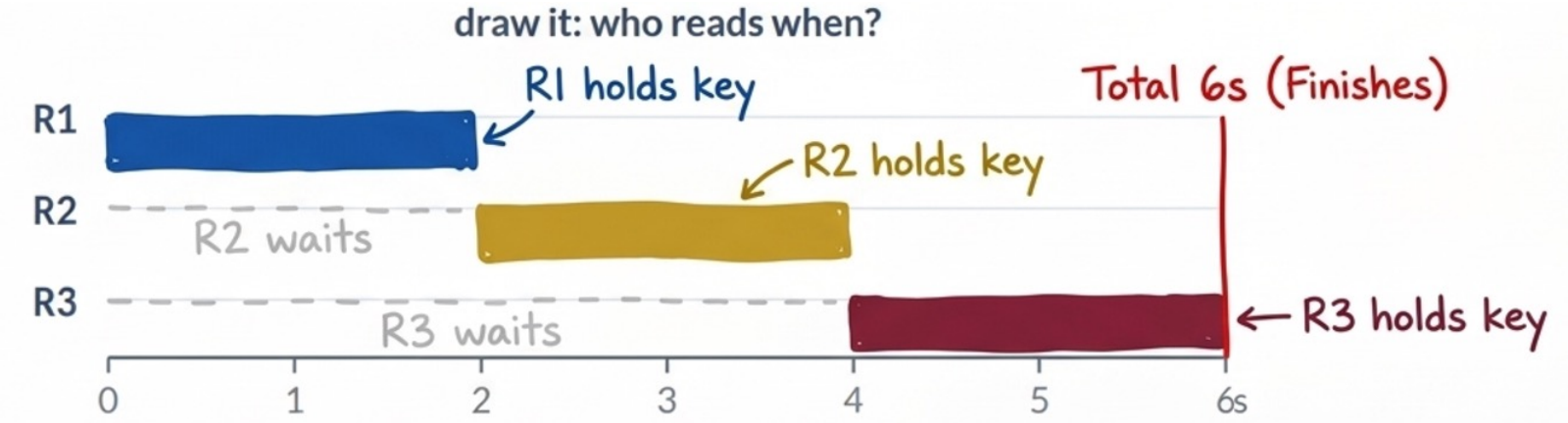
Writer (the cook):       Reader (a customer):
  wait(board)              wait(board)
  rewrite the menu         read the menu
  post(board)              post(board)
```

The rule to keep

Any number of readers, or exactly one writer — never both.

Exercise · on paper

(a) Is it **correct**: can a reader ever be at the board while the cook rewrites it?
(b) Three customers R1, R2, R3 arrive at the same moment; each reads for **2 seconds**; the cook does not come. When has the last one finished?
(c) What did we lose?



First ask whether it is correct; then ask what it costs.

Hint

Draw who holds the key, second by second.

Answer: One Lock, Then the Lightswitch

One lock for the board

(a) Correct: one key, never a reader with the cook. **(b) 6 s**, not 2: R1 reads 0–2 s, R2 2–4 s, R3 4–6 s. **(c)** The readers wait for each other **for nothing**: reading changes nothing.

Lightswitch

Downey's name for *"the first person into a room switches the light on, the last one out switches it off"*: **the first reader in locks the board, the last one out unlocks it.**




```
board   = Semaphore(1)   /* who owns the chalkboard   */
mutex   = Semaphore(1)   /* guards the counter        */
readers = 0              /* shadow counter            */

Writer:                  Reader:
  wait(board)              wait(mutex)
  rewrite the menu         readers = readers + 1
  post(board)              if readers == 1: wait(board)   /* first in: lock the board */
                           post(mutex)
                           read the menu
                           wait(mutex)
                           readers = readers - 1
                           if readers == 0: post(board)   /* last out: unlock it */
                           post(mutex)
```

The first one in locks the door; the last one out unlocks it.

Why `readers` is an ordinary integer

You may not read a semaphore: the readers count themselves in a **shadow counter**, guarded by `mutex`.

The writer did not change: `wait(board)` … `post(board)`.

Is the first reader breaking the rule?

Never sleep holding the key — yet he can sleep on board while holding mutex. Allowed here: whoever holds the board (the cook) never asks for mutex, so no circle of waiting can form. The rule is a safety habit; the real test is always the circle: who waits for whom.

Homework: Trace the Chalkboard

THREE READERS AND THE COOK, IN THIS ORDER

step	who	op	readers	board	mutex	sleeping
0		(start)	0	1	1	—
1	R1	enters				
2	R2	enters				
3	W	wait(board)				
4	R1	leaves				
5	R3	enters				
6	R2	leaves				
7	R3	leaves				
8	W	rewrites the menu				
9	W	post(board)				

enters = the reader's lines down to read the menu;
leaves = the lines after it. Each value column holds the value **after** the step.

One row per step: the values after it, and who is asleep.

Exercise · fill in the table

(a) At which step does W fall asleep, and how many readers are inside then?
(b) Who wakes W, and at which step?
(c) W asked for the board before R3 arrived. Who got in first?

```
board = Semaphore(1)   mutex = Semaphore(1)
readers = 0

Reader:                          Writer:
  /* enters */                     wait(board)
  wait(mutex)                      rewrite the menu
  readers = readers + 1            post(board)
  if readers == 1:
      wait(board)
  post(mutex)
  read the menu
  /* leaves */
  wait(mutex)
  readers = readers - 1
  if readers == 0:
      post(board)
  post(mutex)
```

Starvation, Measured

Correct is not the same as fair.

Starvation

A process that is **never chosen** although the program is correct and everybody else keeps moving. Not a deadlock: the café is busy — only the cook never gets the chalk.



SIX READERS AND ONE COOK, FOR REAL

```
$ ./rw_readers
reads 24, writes 0, writer waited at most 10.0 s
$ ./rw_readers
reads 24, writes 0, writer waited at most 10.0 s
$ ./rw_readers
reads 24, writes 0, writer waited at most 10.0 s
```

the idea: 2 s of reading each, a new reader every second



The arithmetic

The board is unlocked only when readers returns to 0. If a new reader comes in before the last one leaves, it never does: the readers keep the board for ever, and W stays asleep on board.

Correct, and still unfair

The lightswitch never lets the cook and a reader share the board. In **8 of 8** runs (3 shown) the cook **never** rewrote the menu (writes 0). He came out at 1 s and waited up to 10.0 s — he got the board only after closing time (10 s).

The Turnstile Fix

Why it works

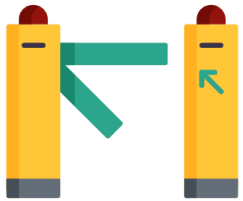
1. A waiting writer takes the turnstile and **keeps it** while it sleep on board: the gate is closed.
2. Readers already inside drain out.
3. New readers queue at the gate.
4. The last one out posts board: the writer writes, then opens the gate.

What the gate costs

Without the gate (8 runs): 24 reads, 0 writes, the cook waits up to 10.0 s. With it (8 runs): 14–18 reads, 2–3 writes, at most 2.0 s — about one reading time (2 s). The price: a little less reading.

```
turnstile = Semaphore(1)     /* the gate at the door */
board     = Semaphore(1)
mutex     = Semaphore(1)
readers   = 0

Writer:
  wait(turnstile)  /* close it */
  wait(board)
  rewrite the menu
  post(turnstile)  /* open it */
  post(board)
```



```
Reader:
  wait(turnstile)  /* pass it */
  post(turnstile)
  wait(mutex)
  readers = readers + 1
  if readers == 1: wait(board)
  post(mutex)
  read the menu
  wait(mutex)
  readers = readers - 1
  if readers == 0: post(board)
  post(mutex)
```

Turnstile

A gate that everybody passes **one at a time**: a semaphore that starts at 1 and is posted straight after it is taken — `wait(turnstile); post(turnstile)`.

```
$ ./rw_turnstile
reads 18, writes 3, writer waited at most 2.0 s
$ ./rw_turnstile
reads 14, writes 2, writer waited at most 2.0 s
$ ./rw_turnstile
reads 18, writes 3, writer waited at most 2.0 s
```

Exercise: Does This One Starve?

Two changes to the turnstile version. Everywhere: turnstile, board and mutex start at 1, readers = 0. The changed lines are in red.

VARIANT A — THE COOK OPENS THE GATE EARLY

```
Writer (A):
  wait(turnstile)
  post(turnstile)
  wait(board)
  rewrite the menu
  post(board)
```

```
Reader:
  wait(turnstile)
  post(turnstile)
  wait(mutex)
  readers = readers + 1
  if readers == 1:
      wait(board)
  post(mutex)
  read the menu
  wait(mutex)
  readers = readers - 1
  if readers == 0:
      post(board)
  post(mutex)
```

VARIANT B — THE READER TAKES THE KEY FIRST

```
Writer:
  wait(turnstile)
  wait(board)
  rewrite the menu
  post(turnstile)
  post(board)
```

```
Reader (B):
  wait(mutex)
  wait(turnstile)
  post(turnstile)
  readers = readers + 1
  if readers == 1:
      wait(board)
  post(mutex)
  read the menu
  wait(mutex)
  readers = readers - 1
  if readers == 0:
      post(board)
  post(mutex)
```

Exercise · for A and for B

(a) Still correct — never a reader at the board with the cook?

(b) Still fair — or can the cook wait for ever while readers come and go?

(c) Which process can sleep for ever, and on which semaphore?

Move one line, and ask the three questions again: correct, fair, alive.

Hint

For every `wait`, ask who holds the flag it needs — and what that holder is waiting for.

Answer: Does This One Starve?

variant	correct?	fair?	who can sleep for ever
A — the gate opens before the cook gets the board	**yes**	**no**: the gate is open while the cook waits, so new readers keep coming in — starvation again	the cook, on `board`, while the readers keep moving (not a deadlock)
B — the reader takes `mutex` before the gate	**yes**	it does not matter: it can **stop**	**everybody**: a deadlock (below)

B, IN FOUR STEPS

step	who	op	readers	board	mutex	turnstile	sleeping
0		(start)	0	1	1	1	—
1	R1	enters, reads	1	0	1	1	—
2	W	wait(turnstile), wait(board)	1	0	1	0	W
3	R2	wait(mutex), wait(turnstile)	1	0	0	0	W, R2 (holding mutex)
4	R1	leaving: wait(mutex)	1	0	0	0	W, R2, R1 — everyone: nobody can move

Who waits for whom in B

W sleeps on `board`: R1 holds it. R1 sleeps on `mutex`: R2 holds it. R2 sleeps on `turnstile`: W holds it. A circle — R2 broke the rule **never sleep holding the key**.

Where you meet it

A database: the readers are queries, the writer is the statement that changes the table's shape. A configuration file re-read while requests are served. A cache with one process that refreshes it.

A lock says who may; fairness says who next.

Two Kinds of Customer, One Bathroom

Categorical exclusion

Whole **categories** keep each other out: the café's one bathroom holds staff or customers, never both — and never more than three people.

Same bricks, composed

Two lightswitches, one per category, over the **same** `room`; inside it, a **limit** of 3. Staff run the mirror image: `mutexS`, `staff`, the same `room` and `limit`.

The order matters

Take the **category first, the place second**. The other order lets a customer keep one of the three places while he sleeps on `room` — staff who arrive then wait for places nobody is using. Holding one thing while waiting for another is how every deadlock starts.

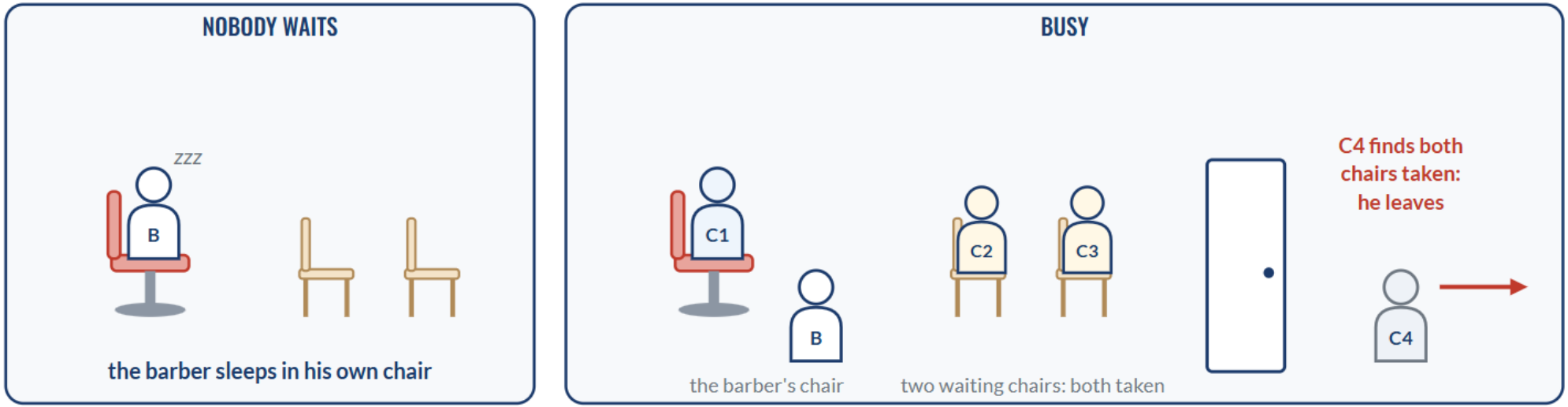
```
room      = Semaphore(1)   /* the empty bathroom            */
limit     = Semaphore(3)   /* three places inside           */
mutexC    = Semaphore(1)   /* guards the customers' counter */
customers = 0              /* shadow counter                */

Customer:
  wait(mutexC)
  customers = customers + 1
  if customers == 1: wait(room)   /* first in: the room is ours */
  post(mutexC)
  wait(limit)                     /* one of the three places    */
  use the bathroom
  post(limit)
  wait(mutexC)
  customers = customers - 1
  if customers == 0: post(room)   /* last out: give it back     */
  post(mutexC)
```

```
Staff:
    wait(mutexS)
    staff = staff + 1
    if staff == 1: wait(room)
    post(mutexS)
```

Two lightswitches share one room; the limit counts the places inside.

The Sleeping Barber



The sleeping barber

Dijkstra's problem, set in the café's corner: **one barber, one barber's chair, two waiting chairs**. Nobody waiting: the barber **sleeps**. A customer who comes in **wakes** him — or sits down and waits.

Balking

Giving up at the door instead of waiting: a customer who finds **both waiting chairs taken leaves** without a haircut.

Three questions the program must answer

1. Who wakes the barber?
2. How does a customer know it is his turn?
3. How do we count the waiting chairs, when a semaphore cannot be read?

Nobody can look inside a semaphore: the barber must be woken, the customer must be called.

The Barber in Pseudo-Code

```
customers = Semaphore(0)   /* "someone is waiting"       */
barber    = Semaphore(0)   /* "next!"                    */
mutex     = Semaphore(1)   /* guards waiting             */
waiting   = 0              /* shadow counter, CHAIRS = 2 */

Customer:                                  Barber:
  wait(mutex)                                loop {
  if waiting == CHAIRS:                        wait(customers)   /* sleep until someone comes */
      post(mutex); leave                       wait(mutex)
  waiting = waiting + 1                        waiting = waiting - 1
  post(mutex)                                  post(mutex)
  post(customers)    /* wake him */            post(barber)      /* "next!" */
  wait(barber)       /* wait for "next!" */    cut hair
  get a haircut                              }
```

0 is a message, 1 is a key

`customers` and `barber` start at **0**: each is a message that somebody **else** gives you. `mutex` starts at **1**: a key you give back **yourself**.

The shadow counter

You may not read `customers` to count the chairs, so the customers count themselves in `waiting` — an ordinary integer, changed only under `mutex`.

The three answers

1. The customer wakes the barber: `post(customers)`. **2.** The barber calls him: `post(barber)`. **3.** `waiting` counts the chairs; a customer who finds it at `CHAIRS` gives the key back and leaves.

Start value 0 is a message; start value 1 is a key.

Homework: Six Customers, Two Chairs

THE ORDER OF EVENTS, AS PRINTED

step	who	op	waiting	customers	barber	sleeping
0		(start)	0	0	0	—
1	Barber	wait(customers)				
2	C1	arrives				
3	Barber	next!				
4	C2	arrives				
5	C3	arrives				
6	C4	arrives				
7	Barber	wait(customers), next!				
8	C5	arrives				
9	Barber	wait(customers), next!				
10	C6	arrives				
11	Barber	wait(customers), next!				
12	Barber	wait(customers), next!				
13	Barber	wait(customers)				

arrives = the customer's lines down to wait(barber) (or to leave);
next! = the barber's four lines after wait(customers).
C2, C3 and C4 arrive while C1 is in the chair.

One row per step: who sleeps, who is called, who leaves.

```
CHAIRS = 2     waiting = 0
mutex     = Semaphore(1)
customers = Semaphore(0)
barber    = Semaphore(0)
Customer:
  wait(mutex)
  if waiting == CHAIRS:
      post(mutex); leave
  waiting = waiting + 1
  post(mutex)
  post(customers)
  wait(barber)
  get a haircut
Barber:
  loop {
    wait(customers)
    wait(mutex)                  ┐
    waiting = waiting - 1  next!
    post(mutex)                  │
    post(barber)                 ┘
    cut hair
  }
```

(a) Who leaves without a haircut?
(b) In which order are the haircuts given?
(c) Swap the customer's lines post(customers) and wait(barber). What happens? (next lecture …)

Answer: Six Customers, and k Barbers

step	who	op	waiting	customers	barber	sleeping
0		(start)	0	0	0	—
1	Barber	wait(customers)	0	0	0	Barber
2	C1	arrives	1	0	0	C1 · Barber wakes
3	Barber	next!	0	0	0	C1 wakes
4	C2	arrives	1	1	0	C2
5	C3	arrives	2	2	0	C2, C3
6	C4	arrives	2	2	0	C2, C3 · C4 leaves
7	Barber	wait(customers), next!	1	1	0	C3 · C2 wakes
8	C5	arrives	2	2	0	C3, C5
9	Barber	wait(customers), next!	1	1	0	C5 · C3 wakes
10	C6	arrives	2	2	0	C5, C6
11	Barber	wait(customers), next!	1	1	0	C6 · C5 wakes
12	Barber	wait(customers), next!	0	0	0	C6 wakes
13	Barber	wait(customers)	0	0	0	Barber

(a) C4 · (b) C1, C2, C3, C5, C6

C4 found `waiting == 2` and left. Every `post(barber)` went straight to the customer asleep longest: `barber` stays 0.

```
$ ./barber
 0 s  C1 sits (1 waiting)
 0 s  C1 gets a haircut
 1 s  C2 sits (1 waiting)
 2 s  C3 sits (2 waiting)
 3 s  C2 gets a haircut
 3 s  C4 sits (2 waiting)
 4 s  C5 leaves: no chair
 5 s  C6 leaves: no chair
 6 s  C3 gets a haircut
 9 s  C4 gets a haircut
```

k barbers, same program

Run the barber's loop in **k** processes, change nothing: `post(barber)` now means “a barber is free”. A **pool**: k workers taking jobs from one queue — Week 5's team of operators.

k barbers, same program: the pool is a barber shop.

Before the Next Lecture

THINK ABOUT IT: THE CUSTOMER'S TWO LINES SWAPPED

```
Customer (two lines swapped):     Barber:
  wait(mutex)                       loop {
  if waiting == CHAIRS:               wait(customers)
      post(mutex); leave              wait(mutex)
  waiting = waiting + 1               waiting = waiting - 1
  post(mutex)                         post(mutex)
  wait(barber)                        post(barber)
  post(customers)                     cut hair
  get a haircut                     }
```

Think about it

Start values: `customers = barber = 0`, `mutex = 1`, `waiting = 0`. One customer arrives. Who sleeps, on which semaphore — and who could ever wake them? Bring a trace table of two rows.

THE SWAPPED SHOP, RUN FOR REAL

```
$ timeout 5 ./barber_swapped; echo "exit code $?"
 0 s  C1 sits (1 waiting)
 1 s  C2 sits (2 waiting)
 2 s  C3 leaves: no chair
 3 s  C4 leaves: no chair
 4 s  C5 leaves: no chair
exit code 124
$ ./barber_swapped > out.txt &
$ sleep 7
$ ps -o pid,ppid,stat,comm
  PID  PPID STAT COMMAND
    1     0 Ss   bash
  129     1 S    barber_swapped
  131   129 S    barber_swapped
  132   129 S    barber_swapped
  133   129 S    barber_swapped
  138     1 R    ps
```

Trace it before you run it: the table tells you what the run will show.

Who waits for whom

The customer waits for "next!" **before** telling the barber he is there; the barber waits for a customer. Each waits for a post that only the other can make — after his own `wait`. A deadlock, on every run.

`timeout` and 124

`timeout 5 cmd` kills the command after 5 seconds; exit code **124** means it had to: the shop would never have closed by itself. State `S`: asleep, using no CPU.



Questions?